



BSOS

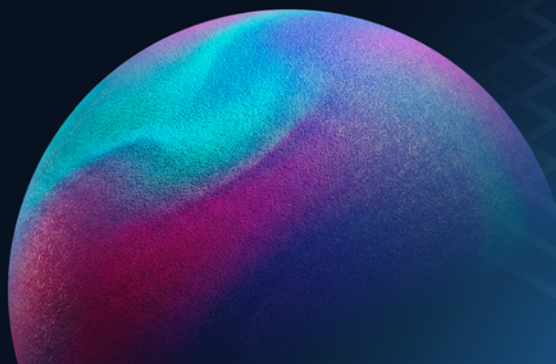
# 智能合約安全檢測報告

檢測工具

**Slither**

檢測日期

**2026-08-14 ~ 2026-08-14**



內部工作版本 — 不可作為交付文件

本報告尚有未完成處理或待確認之項目，僅供工程團隊追蹤使用；請於完成處理後重新產出報告，方可交付。

## 目錄

- 檢測範圍與方法
- 摘要
- 檢測結果

## 檢測範圍與方法

### 掃描範圍

本次檢測涵蓋以下 1 個 Solidity 原始檔（合計 133 行）。此清單即本報告的效力邊界——未列於此的檔案不在本次檢測範圍內。

| 檔案                      | 行數  |
|-------------------------|-----|
| src/VulnerableVault.sol | 133 |

檔案內容雜湊（SHA-256），供比對交付程式碼是否與受檢版本一致：

src/VulnerableVault.sol — 9f47f4a862f5300b7d2dad3322351e41ff098bbe52699ebf263eca64c9dbec0d

### 檢測方法

本次檢測依以下步驟執行：

- 靜態分析掃描**：以 Slither 對「掃描範圍」所列全部原始檔執行完整 detector 掃描。
- 逐筆判讀與補充比對**：對掃描產出的每一筆發現判定其處置分類並記錄判斷依據，工具回報的嚴重度與本報告呈現的嚴重度若有落差則逐筆附上調整理由；同時對範圍內每一份合約，逐條比對內部維護的邏輯漏洞情境庫（權限檢查實作、未保護的狀態變更、旗標未落實、價格源可操縱、記帳與實際結果脫鉤、簽章雜湊綁定範圍、可組合模組的交互失效等），並依受檢系統所屬業務領域比對該領域公開已知的事故模式，補靜態規則無法涵蓋的業務邏輯層級問題。
- 產出與覆核**：彙整為本報告，並對共用同一判斷理由的發現群組進行抽查。

### 範圍限制

本報告由本檢測工具產出，解讀時請留意以下範圍限制：

- 偵測範圍**：靜態分析擅長偵測「程式寫法特徵層級」的問題（如重入模式、tx.origin 授權、弱亂數來源、未檢查的低階呼叫回傳值等）。
- 已知偵測邊界**：業務邏輯層級的問題——例如權限檢查的實作邏輯錯誤、應存在而未實作的保護、經濟模型層面的攻擊（搶跑、滑點）——靜態規則無法窮舉，本工具以情境庫逐合約比對補充涵蓋，但其涵蓋程度不同於系統性審計。
- 文件性質**：本報告為交付前之**自我檢查證明**，證明工程團隊已執行掃描並對每一筆發現完成逐筆判讀；其不構成、亦不取代由獨立第三方執行之完整安全審計。

## 摘要

受檢對象：(測試夾具)

檢測期間：2026-08-14 ~ 2026-08-14

受檢版本：4eeef1e077b74137cc280cec07123b9a7a2a1ad0

本次共提出 23 項發現，依嚴重程度分布如下：

| 嚴重程度          | 筆數 |
|---------------|----|
| Critical      | 2  |
| High          | 3  |
| Medium        | 4  |
| Low           | 7  |
| Informational | 7  |
| 總計            | 23 |

全部發現逐筆列於「檢測結果」，依處置分類分組。

本受檢對象為迴歸測試夾具，不是要交付的產品合約：VulnerableVault 刻意植入了 9+1 個已知漏洞，用來驗證本工具的偵測與判級是否被改壞（完整解答見 VULNERABILITY\_CATALOG.md）。此節的內容僅供版面驗證之用。

### 合約職責

| 合約                  | 職責                                   |
|---------------------|--------------------------------------|
| VulnerableVault.sol | 存提款、批次付款、依 oracle 報價兌換代幣、抽獎，以及數個管理入口 |

### 資產與托管

使用者存入的 ETH 停在 VulnerableVault 自身，以 balances 記帳；獎池另以 rewardPool 記錄。

### 特權角色權限表

| 角色    | 可呼叫                                                 | 後果                        |
|-------|-----------------------------------------------------|---------------------------|
| owner | emergencyWithdraw、topUpRewardPool、batchPay、shutdown | 可轉出合約內資產、可自毀合約            |
| (無保護) | creditBonus、adminSetPaused                          | 實際上任何人皆可呼叫——這正是本夾具植入的漏洞之一 |

**注意：**上表的 owner 保護在本夾具中是**失效的**（onlyOwner 的判斷式恆為真），詳見發現明細。

## 檢測結果

本次共 23 項發現，本章逐筆列出其中 14 項。其餘 9 項為低嚴重度（Low／Informational）且經判定為可接受風險或誤報者，逐筆紀錄保留於工作底稿，可依需要調閱。

| 嚴重度           | 筆數 |
|---------------|----|
| Critical      | 2  |
| High          | 3  |
| Medium        | 4  |
| Low           | 5  |
| Informational | 0  |

| 處置分類       | 筆數 |
|------------|----|
| 已確認需修復 (A) | 13 |
| 待確認 (D)    | 1  |

編號 [H-1] 為嚴重度代碼（C 危急／H 高／M 中／L 低／I 資訊）加該嚴重度內的序號，僅指派給經判定確實成立、需要處置的發現；經查證為誤報或已接受之風險沿用掃描編號。

### 已確認需修復 (A)

#### [C-1] onlyOwner modifier 的權限檢查恆為真

位置：src/VulnerableVault.sol:32-35

說明：require(msg.sender == owner || true) 恆為真，onlyOwner 形同虛設——emergencyWithdraw()/topUpRewardPool()/batchPay() 實際上任何人皆可呼叫

影響：（待補——具體危害與對應的業務損失）

PoC：（待補——攻擊邏輯，或可重現的測試碼）

建議修法：把 require(msg.sender == owner || true) 改為 require(msg.sender == owner)。修正後需重新檢查 emergencyWithdraw/topUpRewardPool/batchPay 三個入口的實際可呼叫者。

#### [C-2] creditBonus() 完全沒有存取控制

位置：src/VulnerableVault.sol:103-107

說明：任何人可呼叫 creditBonus() 幫任意地址無限加值 balances，再透過 withdraw() 提走合約內全部資金

影響：（待補——具體危害與對應的業務損失）

PoC：（待補——攻擊邏輯，或可重現的測試碼）

建議修法：為 creditBonus() 加上 onlyOwner（或專用的 rewards operator 角色），並確認修正後的 modifier 實作本身有效（見 [C-1]）。

#### [H-1] weak-prng

**位置：**src/VulnerableVault.sol:90-100

**說明：**VulnerableVault.pickWinner() (src/VulnerableVault.sol#90-100) uses a weak PRNG: "rand = uint256(keccak256(bytes)(abi.encodePacked(block.timestamp,block.prevrandao))) % depositors.length (src/VulnerableVault.sol#92-94)"

**影響：**(待補 —— 具體危害與對應的業務損失)

**PoC：**(待補 —— 攻擊邏輯，或可重現的測試碼)

**建議修法：**改用 Chainlink VRF 等可驗證隨機源；若必須鏈上取數，至少改為 commit-reveal 兩階段，並禁止在同一交易內完成投注與開獎。

## [H-2] reentrancy-eth

**位置：**src/VulnerableVault.sol:48-56

**說明：**

- Reentrancy in VulnerableVault.withdraw() (src/VulnerableVault.sol#48-56):
- External calls:
  - (ok,None) = msg.sender.call{value: amount}() (src/VulnerableVault.sol#51)
- State variables written after the call(s):
  - balances[msg.sender] = 0 (src/VulnerableVault.sol#53)
- 可跨函式重入的狀態變數共 1 個 (VulnerableVault.balances)，合計可達函式 6 處；完整清單見掃描原始輸出。

**影響：**(待補 —— 具體危害與對應的業務損失)

**PoC：**(待補 —— 攻擊邏輯，或可重現的測試碼)

**建議修法：**改為 checks-effects-interactions：先把 balances[msg.sender] 歸零、再發出外部呼叫；或加上 OpenZeppelin ReentrancyGuard 的 nonReentrant。

## [H-3] suicidal

**位置：**src/VulnerableVault.sol:85-87

**說明：**VulnerableVault.shutdown() (src/VulnerableVault.sol#85-87) allows anyone to destruct the contract

**影響：**(待補 —— 具體危害與對應的業務損失)

**PoC：**(待補 —— 攻擊邏輯，或可重現的測試碼)

**建議修法：**移除 selfdestruct，或改為 onlyOwner + 兩階段確認（提案後經 timelock 才可執行）。合約自毀是不可逆動作，不應存在無保護的入口。

## [M-1] tx-origin

位置：src/VulnerableVault.sol:79-82

說明：VulnerableVault.adminSetPaused(bool) (src/VulnerableVault.sol#79-82) uses tx.origin for authorization: require(bool,string)(tx.origin == owner,not owner) (src/VulnerableVault.sol#80)

影響：（待補 —— 具體危害與對應的業務損失）

PoC：（待補 —— 攻擊邏輯，或可重現的測試碼）

建議修法：改用 msg.sender 判斷呼叫者。tx.origin 在使用者被誘導呼叫惡意合約時會被冒用，不可作為授權依據。

## [M-2] unchecked-lowlevel

位置：src/VulnerableVault.sol:117-121

說明：VulnerableVault.batchPay(address[],uint256) (src/VulnerableVault.sol#117-121) ignores return value by recipients[i].call{value: amountEach}() (src/VulnerableVault.sol#119)

影響：（待補 —— 具體危害與對應的業務損失）

PoC：（待補 —— 攻擊邏輯，或可重現的測試碼）

建議修法：檢查低階呼叫的回傳值 ((bool ok,) = ...; require(ok))，或改用 OpenZeppelin 的 Address.sendValue。迴圈內單筆失敗要明確決定是整批 revert 還是記錄後續處理。

## [M-3] swap() 無滑點保護與 deadline

位置：src/VulnerableVault.sol:125-130

說明：依 oracle 即時報價成交，無 minTokenOut/deadline 參數，可被搶跑 (front-run)

影響：（待補 —— 具體危害與對應的業務損失）

PoC：（待補 —— 攻擊邏輯，或可重現的測試碼）

建議修法：swap() 加上 minTokenOut 與 deadline 參數，由呼叫者指定可接受的最差成交條件；oracle 報價改用 TWAP 或多來源中位數，避免以單點即時價成交。

## [M-4] paused 旗標未被任何函式檢查

位置：src/VulnerableVault.sol:20

說明：adminSetPaused() 會寫入 paused，但 deposit()/withdraw()/safeWithdraw()/swap() 皆未檢查該旗標，暫停機制形同裝飾，事故應變時無法實際凍結資金流

影響：（待補 —— 具體危害與對應的業務損失）

PoC：（待補 —— 攻擊邏輯，或可重現的測試碼）

建議修法：為 deposit()/withdraw()/safeWithdraw()/swap() 加上 whenNotPaused，或直接改用 OpenZeppelin Pausable。只寫不讀的旗標等於沒有暫停機制。



## [L-1] calls-loop

**位置：**src/VulnerableVault.sol:117-121

**說明：**VulnerableVault.batchPay(address[],uint256) (src/VulnerableVault.sol#117-121) has external calls inside a loop: recipients[i].call{value: amountEach}() (src/VulnerableVault.sol#119)

**影響：**(待補 —— 具體危害與對應的業務損失)

**PoC：**(待補 —— 攻擊邏輯，或可重現的測試碼)

**建議修法：**改為 pull 模式（受款方自行領取），或限制單批長度上限並由呼叫者自負 gas；同時處理單筆失敗不應讓整批卡死的情況。

## [L-2] reentrancy-benign

**位置：**src/VulnerableVault.sol:48-56

**說明：**

- Reentrancy in VulnerableVault.withdraw() (src/VulnerableVault.sol#48-56):
- External calls:
  - (ok,None) = msg.sender.call{value: amount}() (src/VulnerableVault.sol#51)
- State variables written after the call(s):
  - totalDeposits -= amount (src/VulnerableVault.sol#54)

**影響：**(待補 —— 具體危害與對應的業務損失)

**PoC：**(待補 —— 攻擊邏輯，或可重現的測試碼)

**建議修法：**與 [H-2] 同一根因，一併以 checks-effects-interactions 修正。

## [L-3] reentrancy-events

**位置：**src/VulnerableVault.sol:90-100

**說明：**

- Reentrancy in VulnerableVault.pickWinner() (src/VulnerableVault.sol#90-100):
- External calls:
  - (ok,None) = winner.call{value: prize}() (src/VulnerableVault.sol#98)
- Event emitted after the call(s):
  - WinnerPicked(winner,prize) (src/VulnerableVault.sol#99)

**影響：**(待補 —— 具體危害與對應的業務損失)

**PoC：**(待補 —— 攻擊邏輯，或可重現的測試碼)

**建議修法：**把事件移到外部呼叫之前發出，或在完成狀態更新後再發出，確保鏈下索引器看到的順序與實際狀態一致。



## [L-4] reentrancy-events

位置：src/VulnerableVault.sol:48-56

說明：

- Reentrancy in VulnerableVault.withdraw() (src/VulnerableVault.sol#48-56):
- External calls:
  - (ok,None) = msg.sender.call{value: amount}() (src/VulnerableVault.sol#51)
- Event emitted after the call(s):
  - Withdrawn(msg.sender,amount) (src/VulnerableVault.sol#55)

影響：（待補 —— 具體危害與對應的業務損失）

PoC：（待補 —— 攻擊邏輯，或可重現的測試碼）

建議修法：同 [L-3]：事件在外部呼叫之後才發出，改為狀態更新完成後立即發出。

### 待確認 (D)

## [L-5] missing-zero-check

位置：src/VulnerableVault.sol:72

說明：

- VulnerableVault.emergencyWithdraw(address,uint256).to (src/VulnerableVault.sol#72) lacks a zero-check on:
  - to.transfer(amount) (src/VulnerableVault.sol#74)

影響：（待補 —— 具體危害與對應的業務損失）

PoC：（待補 —— 攻擊邏輯，或可重現的測試碼）

待確認事項：

- 要確認什麼：emergencyWithdraw 的 to 參數是否可能被傳入零地址：呼叫端是否已在鏈下驗證，或此函式是否僅由多簽觸發。
- 由誰確認：合約擁有者／營運端
- 兩種答案各自的處置：若呼叫端無保證 → 加上 require(to != address(0))；若確認僅由多簽以固定地址呼叫 → 改判 B 並在報告揭露此前提。

## 附錄：發現處置分類

本報告對每一筆發現標示兩個獨立欄位：**嚴重度**（Critical／High／Medium／Low／Informational，由工程團隊依實際影響判定）與**處置**（下列 A／B／C／D）。掃描工具自身回報的 impact 若與本報告呈現的嚴重度不同，該筆會並列印出兩者與調整理由。

- **A | 已確認需修復**：確認為真實問題（多涉及資金流向或權限控制邏輯），必須修復。

- **B | 已知風險但可接受**：問題確實存在，但經工程團隊評估風險可控（例如僅管理者可呼叫、另有其他層級防護），附具體理由後接受並於報告揭露。
- **C | 誤報**：靜態分析限制造成的誤判，實際已有防護機制或該判斷邏輯不適用。
- **D | 待確認**：尚無法判定歸屬者一律列此類；判讀信心不足時寧列 D，不猜測分類。此類項目均附「要確認什麼／由誰確認／兩種答案各自的處置」。